3 장: 판다스 (Pandas) 와 넘파이 (NumPy) –강의 PPT 확장 강의록

교수자: Jung, Minpo (정민포) **교과목**: Machine Learning (기계학습) **학기**: 2026 년도 1 학기 **과정**: 박사과정 수업 **대상**: 중국 박사과정 학생 **주차**: 3 주차 / 3 장: 데이터 처리의 핵심 도구 **수준**: 대학원 박사과정 **강의 시간**: 약 75 분 **키워드**: NumPy, Pandas, ndarray, DataFrame, 벡터화, 브로드캐스팅, 선형대수, 데이터 파이프라인, Tidy Data

---

학습 목표

본 장을 이수한 후 학습자는 다음을 수행할 수 있어야 한다.

1. NumPy ndarray 의 내부 메모리 구조 (stride, dtype, C-order/F-order) 를 설명하고, 성능 최적화에 활용할 수 있다.
2. 브로드캐스팅 규칙을 정확히 적용하여 다양한 형태의 배열 간 연산을 수행할 수 있다.
3. LU/QR/SVD/고유값 분해를 NumPy 로 구현하고, 각 분해가 ML 알고리즘 (PCA, 선형회귀 등) 에서 어떻게 활용되는지 설명할 수 있다.
4. Pandas DataFrame 의 내부 구조를 이해하고, method chaining 과 pipe() 를 사용한 데이터 파이프라인을 구성할 수 있다.
5. 결측치 처리 5 가지 전략, 이상치 탐지 3 가지 방법, 피처 스케일링 3 가지 방법을 비교 분석할 수 있다.
6. Wickham 의 Tidy Data 원칙을 적용하여 messy data 를 tidy data 로 변환할 수 있다.
7. 벡터화 연산과 Python 루프의 성능 차이를 하드웨어 수준 (SIMD, 캐시) 에서 설명할 수 있다.

목차

1. 도입: 데이터 없이는 ML 도 없다
2. NumPy 심화: ndarray 의 세계
3. Pandas 심화: DataFrame 의 세계
4. 데이터 전처리
5. Tidy Data 와 벡터화
6. 논문 리뷰 통합
7. 실습 코드 해설 및 마무리

강의 스크립트 시작

Part 1: 도입–데이터 없이는 ML 도 없다 (약 8 분)

여러분 안녕하세요. 기계학습 박사과정 수업 3 주차에 오신 것을 환영합니다. 오늘은 제가 이 과목 전체에서 실무적으로 가장 중요하다고 생각하는 내용을 다루겠습니다. 바로 데이터 처리의 핵심 도구인 NumPy 와 Pandas 입니다.

본격적으로 들어가기 전에 한 가지 현실적인 이야기를 하겠습니다. Andrew Ng 교수의 유명한 격언이 있죠. "데이터가 왕이다 (Data is King)."이것은 단순한 캐치프레이즈가 아닙니다. 기계학습의 근본적인 진실을 반영하는 말입니다. 아무리 정교한 알고리즘이라 할지라도 양질의 데이터 없이는 유의미한 결과를 산출할 수 없습니다. 그

리고 실무적 함의는 이것입니다. 실제 ML 프로젝트에서 데이터 과학자의 업무 시간 중 **50~80%** 가 데이터 처리에 소비됩니다 (Wickham, 2014). 모델 구축이 아니라 데이터 처리에 말이죠. 그래서 NumPy 와 Pandas 를 숙달하는 것은 선택이 아니라 필수입니다.

슬라이드: ML 워크플로우

전형적인 ML 워크플로우를 보겠습니다.

```
데이터 수집 --> 데이터 탐색(EDA) --> 데이터 전처리 --> 피처 엔지니어링
    --> 모델 학습 --> 모델 평가 --> 배포
```

모델 학습 이전의 모든 단계가 데이터 처리에 해당합니다. NumPy 와 Pandas 는 바로 이 단계들을 위한 핵심 도구입니다.

슬라이드: 파이썬 과학 생태계의 레이어 구조

이제 중요한 그림 하나를 보여드리겠습니다. Harris et al.(2020) 이 Nature 논문에서 제시한 파이썬 과학 생태계의 레이어 구조입니다. 이것은 우리가 왜 이 라이브러리들을 공부하는지 이해하는 데 핵심적입니다.

```
+-----------------------------------------------------+
|              응용 라이브러리 (Applications)            |
|     scikit-learn, TensorFlow, PyTorch, Keras        |
+-----------------------------------------------------+
|           도메인별 라이브러리 (Domain-specific)         |
|     Pandas (표 형식), SciPy (과학 알고리즘)            |
|     matplotlib (시각화), scikit-image (이미지)        |
+-----------------------------------------------------+
|                    NumPy                            |
|       ndarray, ufunc, broadcasting                  |
|       linear algebra, FFT, random                   |
+-----------------------------------------------------+
|            Python + C Extensions                    |
+-----------------------------------------------------+
```

보시는 것처럼 NumPy 는 이 생태계의 **기반 인프라 (foundation infrastructure)** 입니다. Pandas 의 DataFrame 은 내부적으로 NumPy 배열을 감싸고 있고, scikit-learn 의 `fit()` 과 `predict()` 메서드는 NumPy 배열을 입력으로 받습니다. TensorFlow 와 PyTorch 의 텐서도 NumPy 배열과의 상호 변환을 기본 제공합니다. 다시 말해서, NumPy 를 모르면 그 위에 있는 어떤 라이브러리도 제대로 이해할 수 없다는 뜻입니다.

슬라이드: ML 파이프라인에서의 데이터 흐름

실제 코드로 보면 데이터가 어떻게 흐르는지 더 명확해집니다.

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# 1 단계: Pandas 로 데이터 로딩
```

```
df = pd.read_csv('data.csv')

# 2 단계: Pandas 로 탐색 및 전처리
df = df.dropna()
df['new_feature'] = df['feature_a'] * df['feature_b']

# 3 단계: NumPy 배열로 변환
X = df[['feature_a', 'feature_b', 'new_feature']].to_numpy()
y = df['target'].to_numpy()

# 4 단계: scikit-learn 으로 모델링 (내부적으로 NumPy 연산)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
model = LogisticRegression()
model.fit(X_train_scaled, y_train)
```

이 파이프라인에서 확인할 수 있듯이, **Pandas 는 데이터의 로딩, 탐색, 전처리를 담당**하고, **NumPy 는 수치 연산의 기반**을 제공합니다. 그리고 scikit-learn 내부에서도 NumPy 연산이 수행됩니다.

슬라이드: 왜 파이썬인가?

한 가지 더 짚고 넘어가겠습니다. 파이썬이 데이터 과학과 ML 의 사실상 표준 언어가 된 이유의 핵심은 **"접착 언어 (glue language)"**로서의 역할에 있습니다. 파이썬 자체는 느리지만, 성능이 중요한 부분은 C/Fortran 으로 작성된 라이브러리, 예를 들어 BLAS 나 LAPACK 같은 것들이 처리합니다. NumPy 는 이 전략의 대표적인 성공 사례입니다.

McKinney(2010) 는 Pandas 를 개발한 동기를 이렇게 설명합니다. "파이썬에는 R 의 data.frame 에 대응하는 고수준 데이터 구조가 없었다. NumPy 의 ndarray 는 동질적 수치 데이터에는 탁월하지만, 이질적 표 형식 데이터를 다루기에는 부족했다."이 간극을 메우기 위해 Pandas 가 탄생했고, NumPy + Pandas 의 조합은 파이썬을 데이터 과학의 지배적 언어로 만드는 데 결정적 역할을 했습니다.

Part 2: NumPy 심화–ndarray 의 세계 (약 20 분)

자, 이제 본격적으로 NumPy 를 깊이 파고들어 보겠습니다. 넘파이 (NumPy, Numerical Python) 는 파이썬에서 수치 계산을 위한 핵심 라이브러리입니다. 고성능 다차원 배열 (ndarray) 객체와 이를 다루기 위한 다양한 함수를 제공합니다.

```
import numpy as np
```

슬라이드: ndarray 내부 구조–네 가지 핵심 구성 요소

Walt et al.(2011) 이 상세히 기술한 바와 같이, ndarray 는 네 가지 핵심 요소로 구성됩니다.

```
ndarray
+-- data    : 데이터가 저장된 메모리 버퍼의 포인터
+-- dtype   : 각 원소의 데이터 타입 (float64, int32 등)
+-- shape   : 각 차원의 크기를 나타내는 튜플
+-- strides : 각 차원에서 다음 원소까지의 바이트 수
```

여기서 **stride** 가 특히 중요합니다. 이것이 NumPy 의 성능 비결이기도 합니다. 코드로 확인해보겠습니다.

```
arr = np.array([[1.0, 2.0, 3.0],
                [4.0, 5.0, 6.0]])

print(f"dtype:   {arr.dtype}")       # float64
print(f"shape:   {arr.shape}")       # (2, 3)
print(f"strides: {arr.strides}")     # (24, 8)
print(f"nbytes:  {arr.nbytes}")      # 48 = 2 * 3 * 8 bytes
print(f"ndim:    {arr.ndim}")        # 2
print(f"size:    {arr.size}")        # 6
```

stride 가 (24, 8) 이라는 것은 무슨 뜻일까요? 행 방향으로 다음 원소로 이동하려면 24 바이트를 건너뛰어야 하고, 열 방향으로는 8 바이트를 건너뛰어야 한다는 뜻입니다. float64 하나가 8 바이트이니까, 열 방향으로는 바로 옆 원소, 행 방향으로는 3 개 원소 (3 x 8 = 24 바이트) 를 건너뛰는 것입니다.

슬라이드: 메모리 레이아웃–Python 리스트 vs NumPy 배열

ndarray 가 왜 빠른지 이해하려면 메모리 구조를 비교해야 합니다.

Python 리스트의 메모리 구조:

```
리스트 객체 --> [포인터1, 포인터2, 포인터3, ...]
                  |          |          |
                  v          v          v
              PyObject   PyObject   PyObject
              (28+ bytes)(28+ bytes)(28+ bytes)
```

NumPy 배열의 메모리 구조:

```
ndarray 객체 --> [8 bytes | 8 bytes | 8 bytes | ...]
                  값1       값2       값3
                  (연속된 메모리 블록)
```

Python 리스트는 각 원소가 독립적인 Python 객체이고, 리스트는 그 객체들의 포인터만 저장합니다. 그래서 메모리가 불연속적이고, 각 원소에 접근할 때마다 포인터를 따라가야 합니다. 반면 NumPy 배열은 **연속된 메모리 블록**에 동질적 타입의 데이터를 저장합니다. 이것이 CPU 캐시 효율을 극대화하고, SIMD 연산을 가능하게 하는 근본적인 이유입니다.

슬라이드: Stride 와 전치

Stride 의 강력한 점을 보여드리겠습니다.

```
a = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9, 10, 11, 12]], dtype=np.float64)

print(f"shape: {a.shape}")        # (3, 4)
print(f"strides: {a.strides}")    # (32, 8)

# 전치 (transpose) 는 stride 만 바꾼다 (데이터 복사 없음!)
b = a.T
```

```
print(f" 전치 shape: {b.shape}")       # (4, 3)
print(f" 전치 strides: {b.strides}")  # (8, 32)
```

여기서 주목할 점은 전치 연산이 **데이터를 복사하지 않는다**는 것입니다. stride 의 순서만 바꿉니다. 이것은 O(1) 연산입니다. 행렬이 아무리 커도 상수 시간에 전치가 완료됩니다. 이런 설계가 NumPy 를 강력하게 만드는 것입니다.

슬라이드: C-order vs Fortran-order

순서	메모리 레이아웃	빠른 접근 방향	사용 예
C-order (row-major)	행 우선 저장	마지막 축 (열 방향)	NumPy 기본값, C 언어
F-order (column-major)	열 우선 저장	첫 번째 축 (행 방향)	Fortran, MATLAB, R

```
arr_c = np.array([[1, 2, 3], [4, 5, 6]], order='C')
arr_f = np.array([[1, 2, 3], [4, 5, 6]], order='F')

print(f"C-order strides: {arr_c.strides}")    # (24, 8)
print(f"F-order strides: {arr_f.strides}")    # (8, 16)
```

성능 시사점: 행 방향으로 데이터를 순회할 때는 C-order 가, 열 방향으로 순회할 때는 F-order 가 CPU 캐시 효율이 높습니다. NumPy 의 기본값이 C-order 이므로, 일반적으로 행 방향 연산이 더 빠릅니다.

슬라이드: dtype 시스템

카테고리	dtype	바이트	범위/정밀도
정수	`int8`	1	$-128 \sim 127$
정수	`int16`	2	$-32{,}768 \sim 32{,}767$
정수	`int32`	4	$\approx \pm 2.1 \times 10^9$
정수	`int64`	8	$\approx \pm 9.2 \times 10^{18}$
부호 없는 정수	`uint8`	1	$0 \sim 255$ (이미지 픽셀)
부동소수점	`float16`	2	반정밀도 (딥러닝 추론)
부동소수점	`float32`	4	단정밀도 (딥러닝 학습)
부동소수점	`float64`	8	배정밀도 (과학 계산 기본값)
복소수	`complex128`	16	신호 처리, 양자 역학
불리언	`bool_`	1	True/False

dtype 선택은 메모리 효율과 직결됩니다. 예를 들어 딥러닝에서 float32 를 사용하면 float64 대비 메모리를 절반만 사용하면서도 학습 성능에는 거의 차이가 없습니다. 최근에는 float16 이나 bfloat16 까지 사용하는 추세입니다.

슬라이드: 배열 생성 함수 총정리

```
import numpy as np
```

```python
# === 기본 생성 ===
np.array([1, 2, 3])
np.array([[1, 2], [3, 4]], dtype=float)

# === 초기화 생성 ===
np.zeros((3, 4))            # 0 으로 채운 3x4 행렬
np.ones((2, 3))             # 1 로 채운 2x3 행렬
np.full((2, 3), 7)          # 7 로 채운 2x3 행렬
np.empty((3, 3))            # 초기화 없이 메모리 할당
np.eye(4)                   # 4x4 단위행렬
np.diag([1, 2, 3])          # 대각행렬

# === 범위 생성 ===
np.arange(0, 10, 2)         # [0, 2, 4, 6, 8]
np.linspace(0, 1, 5)        # [0, 0.25, 0.5, 0.75, 1.0]
np.logspace(0, 3, 4)        # [1, 10, 100, 1000]

# === 난수 생성 ===
np.random.seed(42)
np.random.rand(3, 4)                        # 균등분포 U(0,1)
np.random.randn(3, 4)                       # 표준정규분포 N(0,1)
np.random.randint(0, 10, size=(3, 4))       # 정수 난수
np.random.normal(0, 1, size=(100,))         # 정규분포
np.random.choice(['a', 'b', 'c'], size=5)   # 임의 선택
```

이 함수들은 여러분이 ML 코드를 작성할 때 매일 사용하게 될 것입니다. 특히 `np.random.seed(42)` 는 재현성을 위해 반드시 설정해야 합니다. 연구 논문에서 실험 결과를 재현하려면 랜덤 시드 고정은 필수입니다.

슬라이드: 배열 인덱싱과 슬라이싱

NumPy 는 네 가지 인덱싱 방법을 지원합니다.

```python
arr = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8],
                [9, 10, 11, 12]])

# 1. 기본 인덱싱 (Basic Indexing)
arr[0, 1]           # 2

# 2. 슬라이싱 (Slicing) -- 뷰 (view) 반환
arr[0:2, 1:3]       # [[2, 3], [6, 7]]
arr[:, 2]           # [3, 7, 11]

# 3. 팬시 인덱싱 (Fancy Indexing) -- 카피 (copy) 반환
arr[[0, 2], [1, 3]] # [2, 12]

# 4. 불리언 인덱싱 (Boolean Indexing)
arr[arr > 5]        # [6, 7, 8, 9, 10, 11, 12]
```

여기서 매우 중요한 구분이 있습니다. 슬라이싱은 **뷰 (view)** 를 반환하고, 팬시 인덱싱과 불리언 인덱싱은 **카피 (copy)** 를 반환합니다. 이 차이를 모르면 버그가 발생할 수 있습니다.

슬라이드: 뷰 (View) 와 카피 (Copy)

```
a = np.array([1, 2, 3, 4, 5])
b = a[1:4]        # 뷰
b[0] = 99
print(a)           # [1, 99, 3, 4, 5] -- 원본도 변경!

c = a[1:4].copy() # 카피
c[0] = 100
print(a)           # [1, 99, 3, 4, 5] -- 원본 불변
```

연산	반환 유형	메모리 공유
슬라이싱 `a[1:4]`	뷰	O
`a.T` (전치)	뷰	O
`a.reshape()`	뷰 (가능 시)	O
팬시 인덱싱 `a[[0,2]]`	카피	X
불리언 인덱싱 `a[mask]`	카피	X
`a.copy()`	카피	X
`a.flatten()`	카피	X
`a.ravel()`	뷰 (가능 시)	O

실무에서 대용량 배열을 다룰 때, 불필요한 카피는 메모리 낭비의 주범이 됩니다. 슬라이싱이 뷰를 반환한다는 것을 활용하면 메모리 효율을 크게 높일 수 있습니다. 하지만 반대로, 뷰를 수정하면 원본도 변경되므로 의도치 않은 데이터 변경에 주의해야 합니다.

슬라이드: 브로드캐스팅 (Broadcasting)

이제 NumPy 의 가장 강력하고도 미묘한 기능인 브로드캐스팅을 다루겠습니다. Walt et al.(2011) 은 이를"크기가 다른 배열 간의 연산을 자동으로 수행하는 메커니즘"으로 정의합니다.

브로드캐스팅의 세 가지 규칙:

규칙 1: 두 배열의 차원 수가 다르면, 차원 수가 적은 배열의 shape **앞에** 1 을 추가한다.

규칙 2: 각 차원에서 크기가 같거나, 둘 중 하나가 1 이면 **호환 (compatible)** 된다.

규칙 3: 크기가 1 인 차원은 다른 배열의 크기에 맞게 **가상으로 확장 (stretch)** 된다.

시각적으로 보겠습니다.

```
예제 1: (3, 3) + (3,)
---
배열 A: shape (3, 3)      배열 b: shape (3,) -> (1, 3) -> (3, 3)

[a00 a01 a02]      [b0 b1 b2]      [a00+b0  a01+b1  a02+b2]
[a10 a11 a12]  +   [b0 b1 b2]  =   [a10+b0  a11+b1  a12+b2]
[a20 a21 a22]      [b0 b1 b2]      [a20+b0  a21+b1  a22+b2]
```

```
예제 2: (3, 1) + (1, 4) -> (3, 4)
---
[a0]                          [a0+b0  a0+b1  a0+b2  a0+b3]
[a1]  +  [b0 b1 b2 b3]  =   [a1+b0  a1+b1  a1+b2  a1+b3]
[a2]                          [a2+b0  a2+b1  a2+b2  a2+b3]


예제 3: 비호환 -- (3,) + (4,) -> ERROR
---
3 != 4 이고 둘 다 1이 아님 -> ValueError
```

ML 에서 브로드캐스팅이 가장 많이 사용되는 패턴은 **데이터 중심화 (centering)** 와 **Z-score 표준화**입니다.

```python
# ML 에서의 활용 -- 데이터 중심화 (centering)
X = np.random.randn(100, 5)
mean = X.mean(axis=0)              # (5,)
X_centered = X - mean              # (100, 5) - (5,) -> 브로드캐스팅

# Z-score 표준화
std = X.std(axis=0)
X_standardized = (X - mean) / std
```

브로드캐스팅의 핵심적 이점은 **실제로 데이터를 복사하지 않는다**는 것입니다. 내부적으로 stride 를 0 으로 설정하여 작은 배열이 큰 배열의 크기에 맞게"가상으로"확장됩니다. 메모리 효율이 극대화되는 것이죠.

슬라이드: 유니버설 함수 (ufunc)

유니버설 함수 (ufunc) 는 배열의 각 원소에 동일한 연산을 적용하는 벡터화된 함수입니다.

분류	함수 예시	설명
수학 함수	np.sin, np.cos, np.exp, np.log, np.sqrt	원소별 수학 연산
비교 함수	np.greater, np.equal, np.logical_and	원소별 비교
산술 함수	np.add, np.subtract, np.multiply	사칙연산
집계 함수	np.sum, np.prod, np.min, np.max	축별 집계

```python
A = np.array([[1, 2, 3],
              [4, 5, 6]])
print(np.sum(A))            # 21 (전체 합)
print(np.sum(A, axis=0))    # [5, 7, 9] (열 방향 합)
print(np.sum(A, axis=1))    # [6, 15] (행 방향 합)
print(np.mean(A, axis=0))   # [2.5, 3.5, 4.5]
```

axis=0 은"행을 따라 축소", 즉 열 방향 합이고, axis=1 은"열을 따라 축소", 즉 행 방향 합입니다. 이 axis 개념은 처음에 헷갈리기 쉬운데, "axis=0 은 행 인덱스가 사라진다"라고 기억하면 됩니다.

슬라이드: 선형대수 연산과 ML 활용

선형대수는 기계학습의 수학적 기반입니다. 이제 핵심적인 행렬 분해들을 NumPy 로 구현하고, 각각이 ML 에서 어떻게 활용되는지 보겠습니다.

기본 행렬 연산

```
A = np.array([[1, 2], [3, 4]], dtype=float)
B = np.array([[5, 6], [7, 8]], dtype=float)

# 원소별 곱셈 vs 행렬곱
print(A * B)        # 원소별 곱셈 (Hadamard product)
print(A @ B)        # 행렬곱 (matrix multiplication)

# 전치, 행렬식, 역행렬, 대각합
print(A.T)
print(np.linalg.det(A))
print(np.linalg.inv(A))
print(np.trace(A))
print(np.linalg.norm(A, 'fro'))
```

`*` 와 `@` 의 차이를 반드시 구분하세요. `*` 는 원소별 곱셈이고, `@` 는 행렬곱입니다. 이 둘을 혼동하면 결과가 완전히 달라집니다.

SVD (특이값 분해) –PCA 의 수학적 기반

$$A = U \cdot \Sigma \cdot V^T$$

SVD 는 ML 에서 가장 중요한 행렬 분해입니다. PCA 차원 축소, 추천 시스템, 잠재 의미 분석 (LSA), 데이터 압축 등에 활용됩니다.

```
A_svd = np.array([[1, 2, 0],
                  [0, 1, 1],
                  [2, 0, 1],
                  [1, 1, 1],
                  [3, 2, 1]], dtype=float)

U, s, Vt = np.linalg.svd(A_svd, full_matrices=False)

# 차원 축소: 상위 k 개 특이값만 사용
k = 2
A_approx = U[:, :k] @ np.diag(s[:k]) @ Vt[:k, :]

error = np.linalg.norm(A_svd - A_approx, 'fro')
original_norm = np.linalg.norm(A_svd, 'fro')
print(f" 정보 보존 비율: {(1 - error / original_norm) * 100:.2f}%")

# 분산 설명력
energy = s ** 2 / np.sum(s ** 2) * 100
print(f" 누적 설명 비율: {np.cumsum(energy)}")
```

SVD 에서 특이값의 크기는 해당 방향의 데이터 분산량을 나타냅니다. 상위 k 개만 사용하면 정보 손실을 최소화하면서 차원을 축소할 수 있습니다. 이것이 바로 PCA 의 수학적 기반입니다. 나중에 PCA 를 배울 때 이 내용이 다시 나올 것입니다.

고유값 분해와 정규방정식

$$A\mathbf{v} = \lambda\mathbf{v}$$

```
cov_matrix = np.array([[4, 2, 1],
                       [2, 3, 1],
                       [1, 1, 2]], dtype=float)

eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)
variance_ratio = eigenvalues / np.sum(eigenvalues) * 100
print(f" 분산 설명 비율: {variance_ratio}")
```

선형회귀의 정규방정식 (Normal Equation) 도 중요합니다.

$$\beta = (X^T X)^{-1} X^T \mathbf{y}$$

```
np.random.seed(42)
n_samples = 50
X_raw = np.random.uniform(0, 10, (n_samples, 1))
noise = np.random.normal(0, 1, (n_samples, 1))
y = 3 * X_raw + 2 + noise

X = np.hstack([np.ones((n_samples, 1)), X_raw])

# 방법 1: 역행렬 (수치적으로 불안정 -- 비추천)
beta_inv = np.linalg.inv(X.T @ X) @ X.T @ y

# 방법 2: solve (수치적으로 안정)
beta_solve = np.linalg.solve(X.T @ X, X.T @ y)

# 방법 3: lstsq (최소제곱법, 가장 안정적, 권장)
beta_lstsq, _, _, _ = np.linalg.lstsq(X, y, rcond=None)

print(f" 참값: 절편 =2, 기울기 =3")
print(f" 추정값: 절편 ={beta_lstsq[0,0]:.4f}, 기울기 ={beta_lstsq[1,0]:.4f}")
```

실무에서는 항상 `np.linalg.lstsq()` 를 사용하세요. 역행렬 계산은 조건수 (condition number) 가 큰 행렬에서 수치적으로 불안정합니다.

슬라이드: 선형대수와 ML 관계 총정리

선형대수 연산	ML 활용	NumPy 함수
SVD	PCA, 추천 시스템, LSA	`np.linalg.svd()`
고유값 분해	PCA, 스펙트럴 클러스터링	`np.linalg.eigh()`
QR 분해	최소제곱법 안정적 풀이	`np.linalg.qr()`

선형대수 연산	ML 활용	NumPy 함수
LU 분해	연립방정식 효율적 풀이	`scipy.linalg.lu()`
정규방정식	선형 회귀	`np.linalg.lstsq()`
행렬곱	신경망 순전파	`@` 연산자
노름	L1, L2 규제	`np.linalg.norm()`

이 표를 잘 기억해두세요. 앞으로 각 알고리즘을 배울 때마다 여기에 있는 선형대수 연산이 반복적으로 등장할 것입니다.

Part 3: Pandas 심화–DataFrame 의 세계 (약 20 분)

이제 Pandas 로 넘어가겠습니다. 판다스 (Pandas) 는 McKinney(2010) 가 금융 데이터 분석을 위해 개발한 라이브러리로, **이질적 (heterogeneous) 표 형식 데이터**를 다루는 데 특화되어 있습니다.

```
import pandas as pd
```

슬라이드: DataFrame 의 내부 구조

DataFrame 은 각 열이 서로 다른 dtype 을 가질 수 있는 2 차원 표 구조입니다. 내부적으로 **각 열은 독립적인 NumPy 배열 (또는 Extension Array)** 로 저장됩니다.

```
df = pd.DataFrame({
    '이름': ['김철수', '이영희', '박민수'],
    '나이': [28, 35, 42],
    '키': [175.5, 162.3, 180.1],
    '학생': [True, False, False]
})

print(df.dtypes)
# 이름      object
# 나이       int64
# 키       float64
# 학생        bool
```

보시는 것처럼 한 DataFrame 안에 문자열, 정수, 실수, 불리언이 공존합니다. 이것이 NumPy ndarray 와의 근본적인 차이입니다. ndarray 는 모든 원소가 동일한 dtype 이어야 하지만, DataFrame 은 열마다 다를 수 있습니다.

슬라이드: 핵심 속성

속성	설명	예시
`df.shape`	(행 수, 열 수)	`(3, 4)`
`df.dtypes`	각 열의 데이터 타입	`int64, float64, ...`
`df.index`	행 인덱스	`RangeIndex(0, 3)`
`df.columns`	열 이름	`Index(['이름', '나이', ...])`
`df.values`	NumPy 배열 변환	`ndarray`

속성	설명	예시
df.info()	전반적 정보	행/열 수, dtype, 메모리
df.describe()	기술통계량	평균, 표준편차, 사분위수

새로운 데이터를 받으면 가장 먼저 df.shape, df.info(), df.describe() 를 실행합니다. 이것이 EDA(탐색적 데이터 분석) 의 첫 단계입니다.

슬라이드: 인덱싱과 선택

Pandas 는 네 가지 주요 인덱싱 방식을 제공합니다.

```
# 1. 열 선택
df['나이']                      # Series 반환
df[['이름', '나이']]            # DataFrame 반환

# 2. loc - 라벨 기반 인덱싱
df.loc[0, '이름']               # '김철수'
df.loc[0:1, ['이름', '나이']]   # 행 0~1, 이름 + 나이 열

# 3. iloc - 정수 위치 기반 인덱싱
df.iloc[0, 0]                   # '김철수'
df.iloc[0:2, 0:2]               # 행 0~1, 열 0~1

# 4. 불리언 인덱싱
df[df['나이'] > 30]             # 나이 > 30 인 행
df.query('나이 > 30 and 키 > 170')  # query 메서드
```

loc vs iloc 의 핵심 차이: loc 은 라벨 (이름) 으로, iloc 은 정수 위치로 접근합니다. 슬라이싱 시 loc 은 끝을 포함하고, iloc 은 끝을 미포함합니다. 이 차이를 반드시 기억하세요.

슬라이드: 메서드 체이닝 (Method Chaining)

Pandas 의 메서드 체이닝은 **중간 변수 없이** 여러 변환을 연속 적용하는 패턴입니다. 현대적인 Pandas 코드에서 가장 권장되는 스타일입니다.

```
# 나쁜 예: 중간 변수 남발
df2 = df.dropna()
df3 = df2[df2['나이'] > 25]
df4 = df3.assign(나이대=df3['나이'] // 10 * 10)
result = df4.sort_values('키', ascending=False)

# 좋은 예: 메서드 체이닝
result = (df
    .dropna()
    .query('나이 > 25')
    .assign(나이대=lambda x: x['나이'] // 10 * 10)
    .sort_values('키', ascending=False)
)
```

두 코드의 결과는 동일하지만, 아래 코드가 훨씬 읽기 좋습니다. 데이터가 어떻게 변환되는지 위에서 아래로 자연스럽게 읽힙니다. pipe() 함수를 사용하면 커스텀 함수도 체이닝에 통합할 수 있습니다.

```
def 이상치 _ 제거 (df, col, n_std=3):
    """Z-score 기반 이상치 제거"""
    mean, std = df[col].mean(), df[col].std()
    return df[abs(df[col] - mean) <= n_std * std]

result = (df
    .pipe(이상치 _ 제거, '키')
    .assign(BMI=lambda x: x['키'] / 100)
)
```

슬라이드: GroupBy –분할-적용-결합 패턴

McKinney(2010) 가 강조한 GroupBy 의 **분할-적용-결합 (Split-Apply-Combine)** 패턴입니다. 이것은 SQL 의 GROUP BY 와 개념적으로 동일하지만, 훨씬 유연합니다.

```
np.random.seed(42)
df = pd.DataFrame({
    '부서': np.random.choice(['개발', '마케팅', '영업'], 100),
    '직급': np.random.choice(['사원', '대리', '과장'], 100),
    '연봉': np.random.normal(5000, 1000, 100).astype(int),
    '성과': np.random.uniform(0, 100, 100).round(1)
})

# agg: 그룹별 집계
df.groupby('부서')['연봉'].agg(['mean', 'std', 'min', 'max'])

# transform: 원본 크기 유지 (그룹별 Z-score)
df['연봉 _zscore'] = df.groupby('부서')['연봉'].transform(
    lambda x: (x - x.mean()) / x.std()
)

# apply: 자유도 높은 그룹별 연산
def 상위 N(group, n=3):
    return group.nlargest(n, '성과')

df.groupby('부서').apply(상위 N, n=2)
```

agg, transform, apply 의 차이를 확실히 구분해야 합니다. agg 는 그룹별로 하나의 값을 반환하여 크기가 줄어들고, transform 은 원본 크기를 유지하면서 그룹별 연산 결과를 각 행에 매핑하며, apply 는 가장 자유도가 높지만 성능은 상대적으로 느립니다.

슬라이드: Merge 와 Join

```
직원 = pd.DataFrame({
    '사번': [1, 2, 3, 4],
    '이름': ['김철수', '이영희', '박민수', '정수진'],
    '부서코드': ['D01', 'D02', 'D01', 'D03']
```

```
})
부서 = pd.DataFrame({
    '코드': ['D01', 'D02', 'D03'],
    '부서명': ['개발팀', '마케팅팀', '영업팀']
})

# SQL JOIN 과 동일
result = pd.merge(직원, 부서, left_on='부서코드', right_on='코드', how='left')
```

Join 유형	SQL 대응	설명
`inner`	INNER JOIN	양쪽 모두 키 있는 행만
`left`	LEFT JOIN	왼쪽 테이블 기준
`right`	RIGHT JOIN	오른쪽 테이블 기준
`outer`	FULL OUTER JOIN	양쪽 모두 포함

SQL 에 익숙하신 분들은 Pandas 의 merge 가 매우 직관적으로 느껴질 것입니다. 실제로 SQL 에서 할 수 있는 거의 모든 데이터 조작을 Pandas 로도 수행할 수 있습니다.

슬라이드: 윈도우 함수 (Window Functions)

시계열 데이터에서 **이동 윈도우 (Rolling Window)** 연산은 핵심적입니다. McKinney(2010) 가 금융 데이터 분석 경험을 바탕으로 Pandas 에 기본 내장한 기능입니다.

```
np.random.seed(42)
dates = pd.date_range('2024-01-01', periods=100, freq='D')
df = pd.DataFrame({
    '날짜': dates,
    '주가': 100 + np.cumsum(np.random.randn(100) * 2),
    '거래량': np.random.randint(1000, 10000, 100)
})
df = df.set_index('날짜')

# 이동 평균 (Moving Average)
df['MA_7'] = df['주가'].rolling(window=7).mean()      # 7 일 이동평균
df['MA_30'] = df['주가'].rolling(window=30).mean()     # 30 일 이동평균

# 이동 표준편차 (변동성 추정)
df['변동성'] = df['주가'].rolling(window=20).std()
```

윈도우 함수	설명	사용 예시
`rolling(n)`	고정 크기 n 윈도우	이동평균, 이동표준편차
`expanding()`	시작 ~ 현재 누적	누적 최대, 누적 평균
`ewm(span=n)`	지수 가중	MACD, 지수이동평균

금융 데이터를 다루시는 분들은 이 윈도우 함수들을 매일 사용하게 될 것입니다. 이동평균 교차 (MA Crossover) 는 기술적 분석의 핵심 신호입니다. 단기 MA 가 장기 MA 를 상향 돌파하면'골든 크로스', 하향 돌파하면'데드 크로스'라고 합니다.

슬라이드: 피벗 테이블과 MultiIndex

```
np.random.seed(42)
매출 = pd.DataFrame({
    '날짜': pd.date_range('2024-01-01', periods=365, freq='D'),
    '지역': np.random.choice(['서울', '부산', '대구'], 365),
    '제품': np.random.choice(['A', 'B', 'C'], 365),
    '매출액': np.random.exponential(100000, 365).astype(int),
    '수량': np.random.randint(1, 50, 365)
})

# 피벗 테이블
피벗 = pd.pivot_table(
    매출,
    values='매출액',
    index='지역',
    columns='제품',
    aggfunc='mean',
    margins=True
)
```

`pivot_table()` 은 SQL 의 `GROUP BY` + `CASE WHEN` 에 대응하는 강력한 데이터 요약 도구입니다. Tidy Data 관점에서 보면, `pivot_table()` 의 결과는 wide format 이고, 필요시 `melt()` 로 다시 long format(tidy) 으로 변환할 수 있습니다.

슬라이드: Pandas 메모리 최적화

대규모 데이터에서 메모리 사용량 최적화는 실무에서 매우 중요합니다.

```
# 1. 수치형 다운캐스팅
df['정수열'] = pd.to_numeric(df['정수열'], downcast='integer')

# 2. Categorical 타입 변환 (반복 문자열에 효과적)
df['지역'] = df['지역'].astype('category')
# 예: 100 만 행 x 3 개 고유값인 문자열 열
# object: ~64MB -> category: ~1MB (64 배 절약!)

# 3. 청크 단위 읽기 (메모리 부족 시)
chunks = pd.read_csv('huge.csv', chunksize=100000)
result = pd.concat([chunk.query('조건 > 0') for chunk in chunks])
```

최적화 전략	방법	절약 효과	적용 상황
다운캐스팅	`int64 -> int32/int16`	50~75%	값 범위가 작은 정수열
Categorical	`object -> category`	90~99%	고유값 수가 적은 문자열
Sparse	`SparseArray`	90%+	대부분 0/NaN 인 데이터
청크 읽기	`chunksize` 파라미터	RAM 제한 내 처리	파일 > RAM

Kaggle 대회에서 자주 등장하는 100GB 급 CSV 파일을 다룰 때, Categorical 변환과 다운캐스팅만으로 메모리 사용량을 1/10 로 줄일 수 있습니다. 박사과정 연구에서도 대용량 데이터를 다룰 일이 많으니 이 최적화 기법들을 반드시 익혀두세요.

Part 4: 데이터 전처리 (약 10 분)

데이터 전처리는 ML 파이프라인에서 가장 시간이 많이 소비되는 단계입니다. 여기서는 결측치 처리, 이상치 탐지, 피처 스케일링 세 가지를 다루겠습니다.

슬라이드: 결측치 처리 전략 5 가지 비교

전략	방법	장점	단점	적합한 상황
행 삭제	`dropna()`	단순함	데이터 손실	결측 비율 $< 5\%$
평균 대체	`fillna(mean)`	분포 유지	분산 과소 추정	정규분포, MCAR
중앙값 대체	`fillna(median)`	이상치 강건	분산 과소 추정	편향 분포
보간	`interpolate()`	연속성 유지	외삽 위험	시계열 데이터
그룹별 대체	`groupby.transform`	그룹 특성 반영	구현 복잡	그룹 간 차이 큰 경우

```
df = pd.DataFrame({
    'A': [1, np.nan, 3, np.nan, 5],
    'B': [10, 20, np.nan, 40, 50],
    '그룹': ['X', 'X', 'Y', 'Y', 'Y']
})

# 전략별 비교
print(" 원본:\n", df['A'].values)
print(" 삭제:", df['A'].dropna().values)
print(" 평균:", df['A'].fillna(df['A'].mean()).values)
print(" 중앙값:", df['A'].fillna(df['A'].median()).values)
print(" 보간:", df['A'].interpolate().values)
print(" 그룹별:", df.groupby('그룹')['A'].transform(
    lambda x: x.fillna(x.mean())).values)
```

어떤 전략을 선택할지는 데이터의 특성에 따라 달라집니다. 결측치가 무작위로 발생한 경우 (MCAR) 에는 삭제나 평균 대체가 적절하고, 시계열 데이터에서는 보간이 가장 자연스럽습니다. 그리고 그룹 간 차이가 큰 경우에는 그룹별 대체가 가장 정확합니다. 예를 들어, 성별에 따라 키의 평균이 다르다면, 전체 평균보다 성별 평균으로 대체하는 것이 더 합리적이겠죠.

슬라이드: 이상치 탐지

IQR 방법

$$\text{이상치 범위}: Q_1 - 1.5 \times IQR < x < Q_3 + 1.5 \times IQR$$

Z-score 방법

$$z = \frac{x - \mu}{\sigma}, \quad |z| > 3 \Rightarrow \text{이상치}$$

```
def iqr_ 이상치 (series):
    Q1, Q3 = series.quantile([0.25, 0.75])
    IQR = Q3 - Q1
    lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
    return (series < lower) | (series > upper)

def zscore_ 이상치 (series, threshold=3):
    z = (series - series.mean()) / series.std()
    return abs(z) > threshold
```

IQR 방법은 비모수적 (non-parametric) 이라 분포 가정이 필요 없고, Z-score 방법은 정규분포를 가정합니다. 데이터의 분포를 먼저 확인한 후 적절한 방법을 선택해야 합니다.

슬라이드: 피처 스케일링

스케일러	수식	특성	사용 상황
StandardScaler	$z = \frac{x-\mu}{\sigma}$	평균 0, 분산 1	정규분포, SVM, 로지스틱
MinMaxScaler	$z = \frac{x-x_{min}}{x_{max}-x_{min}}$	[0, 1] 범위	신경망, 이미지
RobustScaler	$z = \frac{x-Q_2}{Q_3-Q_1}$	중앙값/IQR 기반	이상치 존재 시

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

# 중요: fit 은 train 에만, transform 은 train/test 모두
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)  # fit 없이 transform 만!
```

여기서 매우 중요한 경고를 하겠습니다. **Data Leakage(데이터 유출)** 입니다. `fit_transform` 을 전체 데이터에 적용하면 테스트 데이터의 정보가 훈련에 유출됩니다. 반드시 **훈련 데이터에만 fit**, 테스트 데이터에는 **transform 만** 적용해야 합니다. 이것은 ML 초보자가 가장 많이 범하는 실수 중 하나입니다. 논문 리뷰에서도 이 실수가 발견되면 심각한 결함으로 지적됩니다.

Part 5: Tidy Data 와 벡터화 성능 (약 7 분)

슬라이드: Tidy Data 개념

Wickham(2014) 이 제시한 **깔끔한 데이터 (Tidy Data)** 의 세 가지 원칙입니다.

1. **각 변수는 하나의 열**을 구성한다
2. **각 관측은 하나의 행**을 구성한다
3. **각 관측 단위 유형은 하나의 테이블**을 구성한다

이것이 왜 중요할까요? scikit-learn 이 요구하는"행 = 샘플, 열 = 특성"형식의 입력 데이터가 바로 Tidy Data 원칙과 정확히 일치하기 때문입니다.

```
# Messy: 열 헤더가 값인 경우
messy = pd.DataFrame({
    '이름': ['김철수', '이영희'],
```

```
    '2023_ 국어': [85, 92],
    '2023_ 수학': [90, 88],
    '2024_ 국어': [88, 95],
    '2024_ 수학': [92, 90]
})

# Tidy 로 변환
tidy = (messy
    .melt(id_vars='이름', var_name='과목 _ 연도', value_name='점수')
    .assign(
        연도=lambda x: x['과목 _ 연도'].str.split('_').str[0],
        과목=lambda x: x['과목 _ 연도'].str.split('_').str[1]
    )
    .drop(columns='과목 _ 연도')
)
print(tidy)
```

Tidy Data 의 실무적 이점은 명확합니다. `groupby/agg` 가 자연스럽게 동작하고, seaborn 등 시각화 라이브러리와 호환되며, ML 파이프라인에 바로 투입 가능합니다.

슬라이드: 벡터화 vs 루프–성능의 과학

이제 성능 이야기를 하겠습니다. 왜 NumPy 가 빠른가? 이 표를 보세요.

요인	Python 루프	NumPy 벡터화
실행 엔진	Python 인터프리터	C/Fortran 컴파일 코드
타입 체크	매 연산마다	한 번만
메모리 접근	불연속 (포인터 추적)	연속 (캐시 친화적)
SIMD 활용	불가	가능 (SSE, AVX)
GIL	보유	해제 가능
BLAS/LAPACK	미사용	활용 (Intel MKL 등)

Harris et al.(2020) 의 Nature 논문에서도 확인된 바와 같이, 벡터화는 일반적으로 **100~1000 배** 속도 향상을 가져옵니다.

```
import numpy as np
import time

n = 1_000_000
a = np.random.randn(n)
b = np.random.randn(n)

# Python 루프
start = time.perf_counter()
c_loop = [a[i] + b[i] for i in range(n)]
t_loop = time.perf_counter() - start

# NumPy 벡터화
```

```
start = time.perf_counter()
c_vec = a + b
t_vec = time.perf_counter() - start

print(f"Python 루프: {t_loop:.4f}초")
print(f"NumPy 벡터화: {t_vec:.6f}초")
print(f" 속도 향상: {t_loop/t_vec:.0f}배")
```

100 만 개 원소의 덧셈에서 벡터화가 수백 배 빠릅니다. 이 차이의 원인은 두 가지입니다. 첫째, **SIMD(Single Instruction, Multiple Data)**: 하나의 CPU 명령어로 여러 데이터를 동시에 처리합니다. AVX-256 의 경우 8 개의 float32 를 한 번에 연산합니다. 둘째, **CPU 캐시 효율**: NumPy 배열은 연속 메모리에 저장되어 L1/L2 캐시 적중률이 높습니다. Python 리스트는 포인터를 통한 간접 접근으로 캐시 미스가 빈번합니다.

실용적인 예로, 쌍별 유클리드 거리 계산을 비교해보겠습니다.

```
def 거리_루프(X, Y):
    """Python 3 중 루프: O(n*m*d)"""
    n, d = X.shape
    m = Y.shape[0]
    D = np.zeros((n, m))
    for i in range(n):
        for j in range(m):
            s = 0
            for k in range(d):
                s += (X[i, k] - Y[j, k]) ** 2
            D[i, j] = np.sqrt(s)
    return D

def 거리_브로드캐스팅(X, Y):
    """NumPy 브로드캐스팅: 단 한 줄"""
    return np.sqrt(np.sum((X[:, np.newaxis, :] - Y[np.newaxis, :, :]) ** 2, axis=2))
```

200x300 쌍별 거리 계산에서 벡터화는 루프 대비 약 **500~1000 배** 빠릅니다. KNN, K-Means 등 거리 기반 알고리즘에서 벡터화는 선택이 아니라 필수입니다.

Part 6: 논문 리뷰 통합 (약 5 분)

이번 장과 관련된 핵심 논문 5 편을 간략히 정리하겠습니다.

슬라이드: 핵심 논문 개관

#	논문	핵심 기여	인용수
1	McKinney (2010)	Pandas DataFrame 구조 최초 발표, 분할-적용-결합 패턴	10,000+
2	Walt et al. (2011)	ndarray 내부 구조 (stride, dtype) 상세 문서화	5,000+
3	Harris et al. (2020)	NumPy Nature 논문, 배열 프로그래밍 패러다임 체계화	8,000+

#	논문	핵심 기여	인용수
4	Reback et al. (2020)	Pandas 공식 인용 문서, 전체 기능 개관	15,000+
5	Wickham (2014)	Tidy Data 3 원칙, messy data 5 가지 유형 분류	7,000+

슬라이드: 논문 간 연결

다섯 논문의 관계를 시간순으로 정리하면 이렇습니다.

```
McKinney (2010)    Walt et al. (2011)    Wickham (2014)    Harris et al. (2020)
  Pandas 탄생  <--  ndarray 기반 문서화      Tidy Data 원칙      NumPy 총정리
      |                    |                    |                    |
      v                    v                    v                    v
  DataFrame = ndarray 기반   stride/ufunc/broadcast  melt/pivot 설계 영향   생태계 레이어 시각
```

McKinney(2010) 는"ndarray 는 이질적 데이터에 부족하다"는 문제를 제기하고 Pandas 를 개발했습니다. Walt et al.(2011) 은 그 기반인 ndarray 의 기술적 설계를 문서화했습니다. Wickham(2014) 은 데이터가 분석에 적합한 형태로 정리되어야 한다는 원칙을 제시하여, Pandas 의 `melt()`/`pivot_table()` 설계에 영향을 주었습니다. Harris et al.(2020) 은 이 전체 생태계를 Nature 에서 공식 인정받았습니다.

박사과정 학생으로서 이 논문들을 직접 읽어보시기를 강력히 권합니다. 특히 Harris et al.(2020) 의 Nature 논문은 반드시 읽어야 할 필독 문헌입니다.

Part 7: 실습 코드 해설 및 마무리 (약 5 분)

슬라이드: 실무 데이터 파이프라인–Titanic 예제

마지막으로, 지금까지 배운 모든 것을 하나의 실무 파이프라인으로 통합하겠습니다.

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 데이터 생성 (Titanic 유사)
np.random.seed(42)
n = 891
df = pd.DataFrame({
    'Survived': np.random.binomial(1, 0.38, n),
    'Pclass': np.random.choice([1, 2, 3], n, p=[0.24, 0.21, 0.55]),
    'Sex': np.random.choice(['male', 'female'], n, p=[0.65, 0.35]),
    'Age': np.random.normal(30, 14, n).clip(1, 80),
    'Fare': np.random.exponential(32, n).clip(0, 512),
    'Embarked': np.random.choice(['S', 'C', 'Q'], n, p=[0.72, 0.19, 0.09])
})
```

```python
# 인위적 결측치
df.loc[np.random.choice(n, 177, replace=False), 'Age'] = np.nan

# 전처리 파이프라인 (메서드 체이닝)
df_clean = (df
    .assign(
        Age=lambda x: x.groupby(['Pclass', 'Sex'])['Age']
            .transform(lambda s: s.fillna(s.median())),
        Sex=lambda x: x['Sex'].map({'male': 0, 'female': 1}),
        Embarked=lambda x: x['Embarked'].map({'S': 0, 'C': 1, 'Q': 2})
    )
)

# 피처/타겟 분리
features = ['Pclass', 'Sex', 'Age', 'Fare', 'Embarked']
X = df_clean[features].values
y = df_clean['Survived'].values

# 학습/테스트 분할
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 스케일링 (train 에만 fit!)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 모델 학습 및 평가
model = LogisticRegression(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(f" 정확도: {accuracy_score(y_test, y_pred):.4f}")
```

이 코드에서 오늘 배운 모든 것이 녹아 있습니다. Pandas 로 데이터 로딩 및 전처리, groupby 를 활용한 그룹별 결측치 대체, 메서드 체이닝, NumPy 배열로 변환, StandardScaler 로 스케일링 (train 에만 fit!), 그리고 scikit-learn 으로 모델링. 이것이 실제 ML 프로젝트의 전형적인 흐름입니다.

슬라이드: 보충–np.einsum 과 고급 기법

시간이 허락하면 한 가지 고급 기법을 소개하겠습니다. `np.einsum()` 은 아인슈타인 합산 표기법을 사용하여 다양한 텐서 연산을 단일 함수 호출로 표현합니다.

```python
# 행렬곱: C_ij = sum_k A_ik * B_kj
C = np.einsum('ik,kj->ij', A, B)        # == A @ B

# 배치 행렬곱 (딥러닝 어텐션에서 핵심)
batch_A = np.random.randn(10, 3, 4)
batch_B = np.random.randn(10, 4, 5)
```

```
batch_C = np.einsum('bij,bjk->bik', batch_A, batch_B)

# 자기 어텐션 (Self-Attention) 의 핵심 연산
Q = np.random.randn(8, 64, 32)  # (heads, seq_len, d_k)
K = np.random.randn(8, 64, 32)
scores = np.einsum('hid,hjd->hij', Q, K)  # (heads, seq_len, seq_len)
```

특히 딥러닝에서 Transformer 의 Self-Attention 을 구현할 때 `einsum` 은 매우 유용합니다. 나중에 딥러닝을 다룰 때 이 내용이 다시 나올 것입니다.

슬라이드: 대용량 데이터 처리–Pandas 를 넘어서

Pandas 의 한계도 알아두어야 합니다. 메모리에 전체 데이터를 올려야 하므로 RAM 크기를 초과하는 데이터는 처리할 수 없습니다.

라이브러리	특징	적합한 상황
Dask	Pandas API 호환, 지연 실행, 분산 처리	중-대규모 (수 ~ 수백 GB)
Polars	Rust 기반, 지연 실행, 매우 빠름	단일 머신 대규모
Vaex	Out-of-core, 메모리 매핑, 10 억 행 처리	탐색적 분석
PySpark	클러스터 분산 처리	초대규모 (TB 이상)

박사과정 연구에서 대용량 데이터를 다룰 일이 생기면, 이 라이브러리들을 검토해보세요. 특히 Polars 는 최근 급부상하고 있는 차세대 데이터프레임 라이브러리입니다.

슬라이드: 핵심 요약표

개념	핵심 포인트
ndarray	연속 메모리, stride, 동질적 dtype, 벡터화 연산의 기반
브로드캐스팅	크기 다른 배열 간 연산 자동화, stride=0 가상 확장
DataFrame	이질적 표 구조, 열별 독립 배열, 라벨 기반 인덱싱
메서드 체이닝	`.assign().query().sort_values()` 패턴
GroupBy	분할-적용-결합, `agg`/`transform`/`apply`
Tidy Data	변수 = 열, 관측 = 행, 관측단위 = 테이블 (Wickham 2014)
벡터화	C 루프, 연속 메모리, SIMD 로 100~1000 배 성능 향상
결측치 전략	삭제/평균/중앙값/보간/그룹별–상황에 맞게 선택
스케일링	Standard/MinMax/Robust –train 에만 fit, Data Leakage 주의
SVD	$A = U\Sigma V^T$, PCA 의 수학적 기반, 차원 축소의 핵심

슬라이드: 복습 질문

다음 10 개의 질문에 대해 스스로 답변할 수 있는지 확인해보세요.

1. NumPy ndarray 의 stride 개념을 설명하고, 전치 (transpose) 연산 시 stride 가 어떻게 변하는지 설명하시오.

2. 브로드캐스팅의 세 가지 규칙을 기술하고, (3,1) + (1,4) 연산의 결과 shape 을 유도하시오.
3. C-order 와 F-order 의 차이점을 메모리 레이아웃 관점에서 설명하고, 행 방향 순회 시 어느 것이 더 효율적인지 이유와 함께 설명하시오.
4. Pandas 의 `loc` 과 `iloc` 의 차이를 설명하고, 슬라이싱 시 끝 인덱스 포함 여부가 다른 이유를 설명하시오.
5. McKinney(2010) 가 Pandas 를 개발한 동기를 설명하고, NumPy ndarray 가 표 형식 데이터에 부적합한 이유를 기술하시오.
6. 결측치 처리 5 가지 전략의 장단점을 비교하고, 시계열 데이터에서 가장 적합한 전략은 무엇인지 근거와 함께 설명하시오.
7. Wickham(2014) 의 Tidy Data 3 원칙을 기술하고, 열 헤더가 값인 messy data 를 tidy data 로 변환하는 Pandas 코드를 작성하시오.
8. 벡터화 연산이 Python 루프보다 100~1000 배 빠른 이유를 하드웨어 수준 (SIMD, CPU 캐시) 에서 설명하시오.
9. Harris et al.(2020) 의 Nature 논문에서 제시한 파이썬 과학 생태계의 레이어 구조를 설명하고, NumPy 가 기반 계층에 위치하는 이유를 설명하시오.
10. Data Leakage 란 무엇이며, `StandardScaler` 를 잘못 사용했을 때 어떻게 발생하는지 구체적으로 설명하시오.

오늘 수업을 마치겠습니다. 3 장은 양이 많지만, 여기서 배운 NumPy 와 Pandas 는 앞으로 모든 장에서 반복적으로 사용됩니다. 특히 데이터 전처리와 벡터화는 실무에서 가장 중요한 기술입니다. 다음 주에는 4 장 데이터 시각화를 다루겠습니다. 질문이 있으면 지금 하시거나, 이메일로 보내주세요. 수고하셨습니다.

참고 문헌

1. McKinney, W. (2010). "Data Structures for Statistical Computing in Python."*SciPy 2010*, pp. 56-61. DOI: 10.25080/Majora-92bf1922-00a
2. Walt, S. et al. (2011). "The NumPy Array: A Structure for Efficient Numerical Computation." *Computing in Science & Engineering*, 13(2), 22-30. DOI: 10.1109/MCSE.2011.37
3. Harris, C. R. et al. (2020). "Array programming with NumPy."*Nature*, 585, 357-362. DOI: 10.1038/s41586-020-2649-2
4. Reback, J. et al. (2020). "pandas-dev/pandas: Pandas."*Zenodo*. DOI: 10.5281/zenodo.3509134
5. Wickham, H. (2014). "Tidy Data."*Journal of Statistical Software*, 59(10). DOI: 10.18637/jss.v059.i10